Инструмент автоматизации проектирования

архитектуры микросервисных систем

AutoSystemDesign

Дичковский Владимир

Что делает AutoSystemDesign

Проектирование

- извлекает функциональные и нефункциональные требования из BRD;
- строит типизированную C4-модель компонентов;
- формирует Sequence-диаграмму для каждого требования;
- проектирует OpenAPI 3.1 и Swagger UI.

Проверка и анализ

- проверяет граф, связи, протоколы и трассировку требований;
- исправляет структурные ошибки через ограниченный repair loop;
- оценивает загрузку, задержки и критический путь;
- сохраняет редактируемые и машиночитаемые артефакты.

LLM предлагает модель; программные валидаторы определяют, можно ли передать её следующей стадии.

Единый пример: сервис заказов интернет-магазина

Функциональные требования

- **FR-001**: регистрация заказа по списку товаров с возвратом номера;
- **FR-002**: получение текущего статуса заказа по номеру.

Граница примера

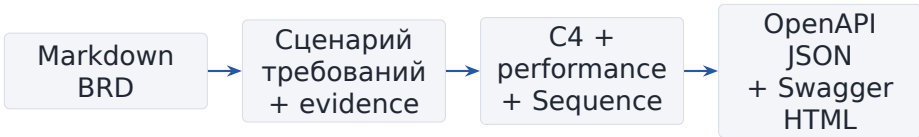
- оплата и управление складом не входят в систему;
- один BRD используется на всех следующих слайдах.

Нагрузка и ограничения

- 120 запросов/с обычно;
- 400 запросов/с в пике;
- 80% чтений, 20% записей;
- p95 ответа API $\leq$ 300 мс;
- бюджет $\leq$ 500 USD/месяц.

Источник: `examples/requirements/online_store_brd.md`.

Вход и результат одного запуска



Основной пользовательский вход

- обычный Markdown-документ BRD;
- YAML-сценарий создаётся автоматически;
- профили производительности необязательны.

Артефакты для разных задач

- PNG/PDF/HTML — просмотр и обсуждение;
- PlantUML — редактирование диаграмм;
- JSON/YAML — интеграция и повторная обработка.

Минимальный запуск из BRD

```
uv run python scripts/generate_from_brd.py \
  --document examples/requirements/online_store_brd.md
```

- результат создаётся в `output/online_store_brd/`;
- сервер и формат диаграмм берутся из конфигурации;
- `--profiles` нужен только для явных измеренных параметров;
- расширенные параметры CLI остаются доступны, но не нужны для первого запуска.

Пользователю не требуется заранее заполнять технические JSON-файлы.

Как BRD превращается в сценарий

BRD: требования, числа и ограничения

↓

LLM выделяет FR, NFR и точные исходные цитаты

↓

Программа сверяет цитаты с исходным документом

↓

Проверенные значения сопоставляются с типизированной моделью сценария

↓

scenario.yaml + evidence.json передаются в конвейер

Проверка происхождения требований

Что сохраняется

- идентификатор требования;
- извлечённое значение;
- цитата из BRD;
- позиция и результат проверки.

Правила

- цитата должна буквально присутствовать в документе;
- числовое значение должно встречаться в подтверждающем фрагменте;
- неоднозначности остаются в diagnostics для просмотра человеком.

Зачем это архитектору: можно проследить, из какого места BRD возникли нагрузка, бюджет или функциональный сценарий, не перечитывая весь документ.

Каталог результата: 16 связанных артефактов

```
output/online_store_brd/
|-- requirements.yaml
|-- requirements.evidence.json
|-- run.json
`-- artifacts/
    |-- online_store_brd_c4.json
    |-- online_store_brd_c4.puml
    |-- online_store_brd_c4.png
    |-- online_store_brd_nfr_coverage.json
    |-- online_store_brd_performance_analysis.json
    |-- online_store_brd_performance_analysis.pdf
    |-- online_store_brd_roles.json
    |-- online_store_brd_seq_fr_001_ast.json
    |-- online_store_brd_seq_fr_001.puml
    |-- online_store_brd_seq_fr_001.png
    |-- online_store_brd_seq_fr_002_ast.json
    |-- online_store_brd_seq_fr_002.puml
    |-- online_store_brd_seq_fr_002.png
    |-- online_store_brd_api_design.json
    |-- online_store_brd_api_openapi.json
    `-- online_store_brd_api_swagger.html
```

Имена артефактов привязаны к одному сценарию; `run.json` содержит итог и статусы стадий.

Архитектура конвейера генерации



LLM отвечает за предложение

- получает схему типизированной модели;
- формирует структурированный JSON;
- не пишет PlantUML напрямую.

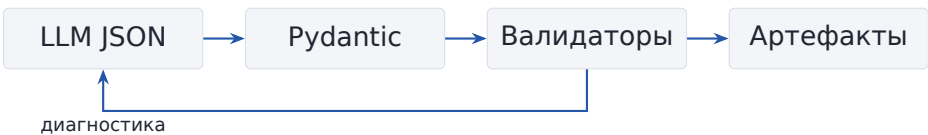
Код отвечает за контракт

- Pydantic разбирает результат;
- доменные спецификации проверяют правила;
- рендереры создают PlantUML, PNG, PDF и HTML.

Стадии связаны типизированным контекстом: последующие артефакты используют проверенные результаты предыдущих.

Repair loop: исправление структурного ответа

1. LLM формирует JSON по схеме доменной модели.
2. Ответ разбирается и передаётся доменным валидаторам.
3. Ошибка JSON или нарушенное правило преобразуется в конкретную диагностику.
4. Диагностика добавляется в следующую попытку генерации.
5. После успешной проверки создаются артефакты; число попыток ограничено.



Доменная модель C4

Типы узлов

- клиент;
- балансировщик;
- сервис;
- база данных;
- кэш;
- брокер сообщений.

Параметры

- реплики;
- параллелизм;
- каналы обслуживания;
- время обработки;
- стоимость.

Связи

- REST/HTTP;
- gRPC;
- database TCP;
- message queue;
- вероятность и режим маршрутизации.

Эта модель одновременно служит источником для C4-диаграммы, расчёта производительности, Sequence и API.

Проверки C4 до построения диаграммы

Структура графа

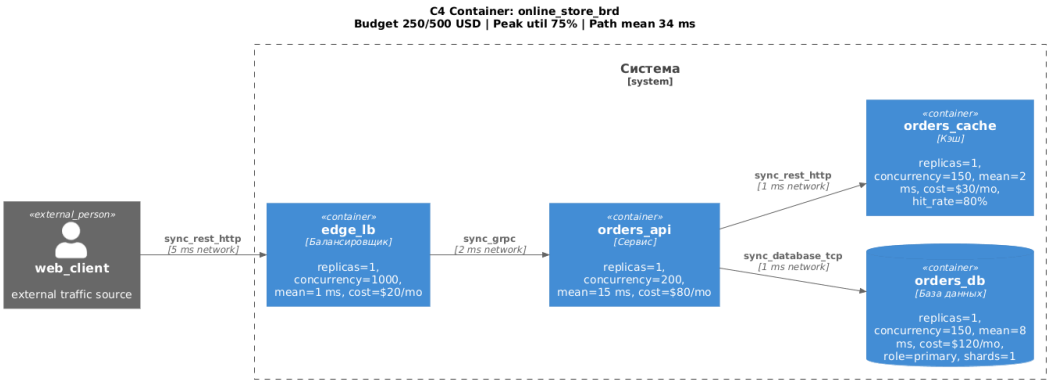
- уникальные идентификаторы узлов;
- существующие концы каждой связи;
- допустимая пара типов узлов;
- отсутствие циклов и изолированных узлов.

Архитектурная семантика

- кэш имеет путь к основному хранилищу;
- брокер связан с producer и consumer;
- протокол согласован с типами компонентов;
- параметры достаточны для расчёта нагрузки.

Онтология связей реализована в коде как набор спецификаций, а не как текстовая рекомендация для LLM.

C4-модель, полученная из BRD



Горизонтальное представление фактической C4-модели: параметры узлов сохранены, изменена только компоновка для слайда.

Sequence-диаграммы связаны с C4

Правила генерации

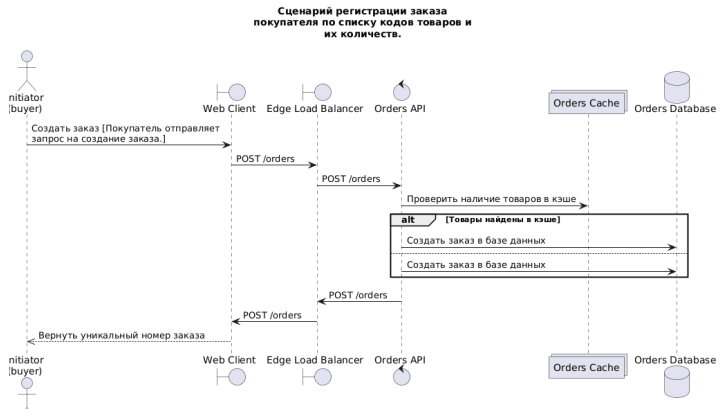
- отдельный сценарий на каждое FR;
- участники ссылаются на роли и узлы C4;
- переходы должны существовать в архитектурном графе;
- протокол влияет на вид сообщения;
- ветвления описываются типизированными блоками.

Проверки

- все объявленные линии жизни используются;
- инициатор и бизнес-роль согласованы;
- события и уведомления асинхронны;
- идентификатор требования сохраняется для API.

Sequence не является независимой картинкой: она проверяется относительно уже принятой C4-модели.

FR-001: регистрация заказа



Путь запроса: покупатель → Web Client → Edge Load Balancer → Orders API → cache/database.

Проектирование API из требований и Sequence

На вход стадии API

- FR-001 и FR-002;
- проверенные Sequence-сценарии;
- компоненты и точки входа C4.

На выходе

- дизайн операций и схем в JSON;
- спецификация OpenAPI 3.1;
- автономная страница Swagger UI.

Дополнительные поля трассировки

- `x-requirement-ids` — исходные функциональные требования;
- `x-sequence-ids` — сценарии поведения;
- `x-entrypoint-component` и `x-handler-component` — узлы C4.

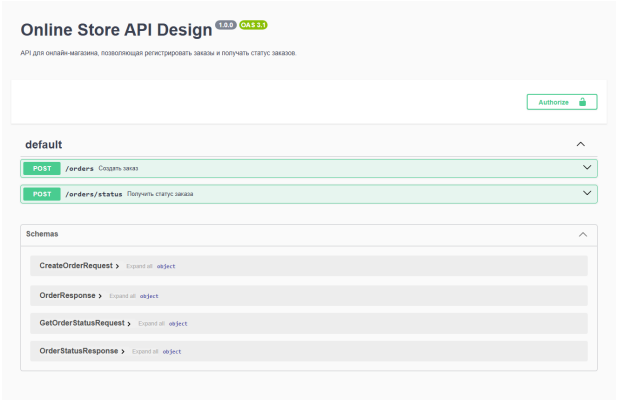
Фрагмент фактического OpenAPI 3.1

```
"/orders": {
  "post": {
    "operationId": "create_order",
    "x-entrypoint-component": "edge_lb",
    "x-handler-component": "orders_api",
    "x-requirement-ids": ["FR-001"],
    "x-sequence-ids": ["SC-FR-001"],
    "responses": {"201": {...}}
  }
}
```

```
"/orders/status": {
  "post": {
    "operationId": "get_order_status",
    "x-entrypoint-component": "edge_lb",
    "x-handler-component": "orders_api",
    "x-requirement-ids": ["FR-002"],
    "x-sequence-ids": ["SC-FR-002"],
    "responses": {"200": {...}}
  }
}
```

Спецификация также содержит request/response schemas и ссылки `$ref` на компоненты OpenAPI.

Swagger UI из того же запуска



HTML открывается локально: операции `POST /orders` и `POST /orders/status`, а также четыре схемы данных.

Как рассчитывается производительность

Распространение потока

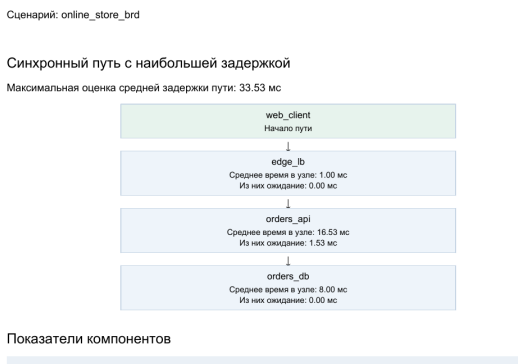
- пиковый RPS проходит по синхронным связям;
- ветви учитывают вероятность и routing mode;
- cache hit rate разделяет быстрый путь и обращение к БД.

Расчёт по узлам

- ёмкость = реплики × каналы / время обслуживания;
- вычисляются utilization, ожидание и средний ответ;
- самый длинный синхронный путь выделяется отдельно.

Для быстрого запуска параметры берутся из C4-кандидата. Если есть измерения, `--profiles` подменяет их явным воспроизводимым каталогом.

PDF: критический путь и загрузка компонентов



Результат примера

- критический путь проходит через четыре узла: `web_client` → `edge_lb` → `orders_api` → `orders_db`;
- оценка среднего пути: **33,53 мс**;
- наибольшая загрузка: **75%** у `orders_api`;
- ниже в PDF приведена таблица всех компонентов.